

Rapport de stage

Étudiant : Oscar-kevine ONDZIGUI-NGUEMA

École : MEDIASCHOOL – IRIS Paris

Période de stage : Du 1 Décembre 2025 au 2 Janvier 2026

Entreprise d'accueil : UP2DATE-IT

REMERCIEMENTS

Je tiens à exprimer ma gratitude envers mon tuteur de stage pour m'avoir accueilli au sein de l'entreprise **UP2DATE-IT** et pour m'avoir accompagné tout au long de ces cinq semaines. Je le remercie pour sa disponibilité, sa pédagogie et le temps qu'il m'a consacré afin de m'expliquer son métier ainsi que les enjeux du monde professionnel.

Je souhaite également remercier l'ensemble des collaborateurs avec lesquels j'ai eu l'opportunité d'échanger, d'assister à des réunions et d'observer le travail en équipe. Cette expérience m'a permis de mieux comprendre la dynamique d'une entreprise et d'acquérir des connaissances utiles pour la suite de mon parcours.

INTRODUCTION

Dans le cadre de ma formation en **BTS Services Informatiques aux Organisations (SIO)**, option SLAM, j'ai effectué un stage de cinq semaines au sein de l'entreprise **UP2DATE-IT**. Ce stage s'inscrit dans la continuité de mon parcours et a pour objectif de mettre en pratique les compétences acquises durant ma formation, tout en découvrant le fonctionnement d'une entreprise spécialisée dans le domaine informatique.

Cette expérience m'a permis de participer à des projets concrets, d'approfondir mes connaissances techniques et de développer des compétences professionnelles essentielles, telles que la communication, la gestion des priorités et le travail en équipe. Elle constitue une étape importante dans mon apprentissage, en me confrontant aux réalités du monde professionnel et en m'offrant une vision plus précise des attentes du secteur.

Présentation de l'entreprise

L'entreprise dans laquelle j'ai effectué mon stage s'appelle **UP2DATE-IT** et a été créée en 2020 par son président actuel.

UP2DATE-IT est une entreprise du **secteur tertiaire**, spécialisée dans le conseil en **systèmes et logiciels informatiques**. Elle propose des services informatiques aux entreprises afin d'améliorer leurs outils numériques et leurs performances.

L'entreprise est une SASU (Société par Actions Simplifiée à Associé Unique). Elle est située au **6 allée Louise Bourgeois à Villejuif**, à proximité du métro Villejuif Léo Lagrange (ligne 7).

UP2DATE-IT compte un salarié, en plus de son président.

L'entreprise travaille principalement avec des ESN (Entreprises de Services du Numérique) en tant que sous-traitant, intervenant sur des missions informatiques pour le compte de leurs clients finaux. Lors de mon stage, UP2DATE-IT travaillait pour la société Sopra-Stéria en tant que sous-traitant chez Malakoff-Humanis.

Même si l'entreprise est de petite taille, elle est rattachée à la convention collective **SYNTEC** et dispose d'une mutuelle d'entreprise.

UP2DATE-IT sous-traite sa comptabilité à une société de comptabilité qui a pour rôle de:

- Enregistrement des factures clients
- Suivre les encaissements et décaissements
- Calculer la TVA collectée
- Calculer la TVA déductible
- Établir des bulletins de paie
- Gérer des avantages (frais, titres-restaurants, etc.)
- Déclaration de l'IS (impôt sur les sociétés)
- Établissement (bilan, compte de résultat)
- etc...

Le grade d'ingénieur informatique

Un ingénieur en informatique est titulaire d'un diplôme d'ingénieur (Bac +5, grade Master) délivré par une école accréditée par la CTI (commission des titres des ingénieurs). C'est pourquoi il est important de choisir une école accréditée par la CTI pour obtenir le grade d'ingénieur. Un ingénieur informatique conçoit, développe, améliore et sécurise des logiciels, des sites internet ou des systèmes informatiques. Il peut également travailler dans la cybersécurité, les réseaux, les bases de données ou l'intelligence artificielle. Outre ces métiers techniques, l'ingénieur en informatique peut aussi avoir le rôle de DSI (Directeur des Systèmes d'Information), Chef de projet informatique (CP / CPI), Responsable portefeuille projets IT, Manager d'équipe IT.

Les parcours pour devenir ingénieur

Pour accéder à un cycle d'ingénieur et afin d'obtenir un diplôme il est nécessaire au préalable:

- Réussir les classes préparatoires des écoles, ou obtenir un BUT (Bachelor Universitaire de Technologie). Dans ces cas le type de bac à réussir est:

A. un bac général

B. ou un bac avec les spécialités combinées Mathématique, NSI (Numérique et Sciences Informatiques), physique-chimie

C. Sciences de l'Ingénieur (SI)

- ou une licence (Mathématique, Physique, Science de l'Ingénieur ou licence d'informatique)

Parcours de mon tuteur

Mon tuteur a obtenu un baccalauréat C, il y a 25 ans. Ce bac serait l'équivalent aujourd'hui d'un baccalauréat général avec les spécialités mathématiques, physique-chimie. Il a ensuite intégré une faculté de science, spécialité Maths-Physique, avant d'immigrer en France où il a été admis en deuxième année de prépa-intégré de l'école esiea. A l'issue de cette deuxième année de classes préparatoires, il a intégré le cycle ingénieur de l'école et a obtenu son diplôme d'ingénieur esiea. Après l'obtention de son diplôme d'ingénieur, il a travaillé en tant que consultant dans plusieurs ESN(Entreprise de Services du Numérique) dans lesquelles il a occupé différents postes (développeur, architecte, chef de projet MOE). Aujourd'hui, il travaille en tant que consultant senior indépendant dans diverses sociétés qui ont besoin de renforcer leurs équipes techniques.

Table des matières

1 Présentation de l'architecture applicative	2
1.1 Vue d'ensemble	2
1.2 Architecture Front-end (Angular)	2
1.2.1 Organisation modulaire	2
1.2.2 Gestion de l'état applicatif (NgRx)	3
1.2.3 Backend et services externes (Firebase)	3
1.2.4 Communication entre les couches	3
1.2.5 Avantages de l'architecture	4
2 Migration Angular 12 vers Angular 21	4
2.1 Pourquoi une migration progressive a été nécessaire	4
2.2 Utilisation du guide officiel Angular	4

2.3 Pattern général utilisé à chaque migration (ex. Angular N → Angular N+1) 6

2.4 Apport de l'intelligence artificielle dans la migration 8

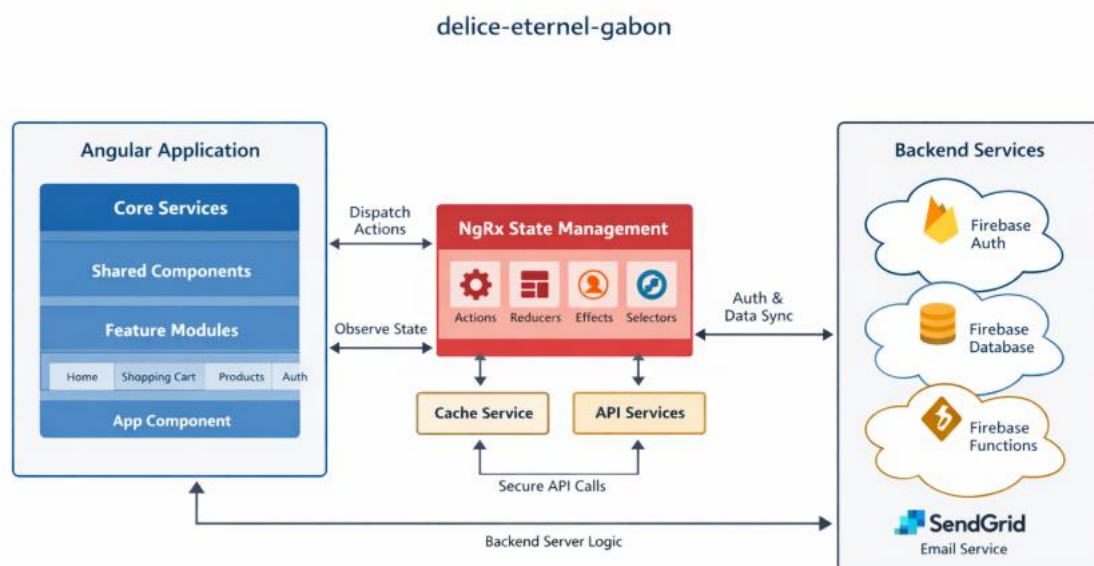
2.5 Firebase et notion de SDK modulaire 8

2.6 Environnement technique et outils utilisés 9

2.7 Conclusion 10

3 Lexique détaillé du projet 11

1 Présentation de l'architecture applicative



1.1 Vue d'ensemble

L'application delice-eternel-gabon repose sur une architecture web moderne de type SPA (Single Page Application) développée avec le framework Angular.

Elle est conçue selon une architecture modulaire permettant une séparation claire des responsabilités entre l'interface utilisateur, la logique métier, la gestion de l'état et les services backend. Cette organisation garantit une application maintenable, évolutive et performante.

1.2 Architecture Front-end (Angular)

1.2.1 Organisation modulaire

L'application Angular est structurée en plusieurs couches distinctes :

Core :

Contient les services globaux tels que l'authentification, les services API, le cache, les guards de routes et les interceptors HTTP.

Ces services sont chargés une seule fois au démarrage de l'application.

Shared :

Regroupe les composants, services, modèles et pipes réutilisables dans l'ensemble de l'application. Cette couche permet d'éviter la duplication de code.

Features :

Chaque fonctionnalité métier (accueil, panier, produits, contact, profil utilisateur) est isolée dans son propre module. Cette organisation facilite la maintenance et l'évolution du projet.

Auth :

Module dédié à l'authentification des utilisateurs via Firebase (Google, email/mot de passe).

1.2.2 Gestion de l'état applicatif (NgRx)

L'application utilise NgRx pour gérer l'état global de manière centralisée. L'idée est que toutes les variables applicatives soient enregistrées au même endroit dans l'application. Cet endroit s'appelle le store et est lié au local storage du navigateur.

Rolde de NgRx

NgRx permet :

la centralisation de l'état de l'application,

la synchronisation des données avec Firebase,

une gestion prévisible et cohérente des données entre les différentes pages de l'application.

Fonctionnement globale

Actions : déclenchées par les composants angular lors des interactions utilisateur.

Reducers : responsables de la mise à jour de l'état.

Selectors : utilisés pour lire les données depuis le store.

Effects : gèrent les opérations asynchrones telles que les appels Firebase ou bien la transmission des données d'un composant à un autre.

1.2.3 Backend et services externes (Firebase)

Le backend repose sur Firebase de google, ce qui permet une infrastructure légère et économique.

Firebase Authentication

Gère l'authentification des utilisateurs de manière sécurisée.

Firebase Realtime Database / Firestore

Stocke les données des utilisateurs et des commandes avec une synchronisation en temps réel.

Firebase Functions

Permet l'exécution de logique serveur, notamment l'envoi d'emails via SendGrid.

1.2.4 Communication entre les couches

Le flux de données suit le schéma suivant :

Interaction utilisateur via l'interface Angular

Déclenchement d'une action NgRx

Traitement via les services ou effects

Communication avec Firebase

Mise à jour automatique de l'interface

1.2.5 Avantages de l'architecture

- Séparation claire des responsabilités
- Maintenance facilitée
- Scalabilité de l'application
- Sécurité renforcée
- Compatibilité avec les évolutions futures d'Angular

Cette architecture n'a pas été modifiée dans son ensemble. Seuls les composants logiciels (bibliothèques externes) ont été migrés vers des versions plus récentes. Le code source de l'application a été adapté à la migration avec de légères modifications. Cette migration a constitué une étape clé du projet afin de répondre aux nouvelles exigences d'Angular 21. Le paragraphe suivant décrit en détail le travail de migration qui a été fait sur le projet.

2 Migration Angular 12 vers Angular 21

Ce rapport présente le travail de migration d'une application web développée initialement avec Angular 12 vers Angular 21. Elle avait pour objectif de moderniser l'application, d'améliorer ses performances, de renforcer sa maintenabilité et de la rendre compatible avec les évolutions récentes de l'écosystème Angular.

2.1 Pourquoi une migration progressive a été nécessaire

La migration n'a pas pu être effectuée directement d'Angular 12 vers Angular 21. Chaque version majeure d'Angular introduit des changements importants, tels que la dépréciation de certaines API, l'apparition de nouveaux concepts (composants standalone, SDK modulaires) et des mises à jour de dépendances.

La migration a donc été réalisée progressivement : Angular 12 → 13 → 14 → 15 → 16 → 17 → 18 → 19 → 20 → 21. Cette approche a permis de corriger les erreurs étape par étape. Par exemple, les erreurs liées à NgRx (@Effect déprécié) ont été résolues en migrant vers createEffect, et les conflits de dépendances npm ont été traités progressivement.

2.2 Utilisation du guide officiel Angular

Le guide officiel <https://angular.dev/update-guide> a été utilisé tout au long de la migration. Il fournit des recommandations précises pour chaque version d'Angular, en indiquant les commandes à exécuter, les dépendances à mettre à jour et les changements de comportement à anticiper.

2.3 Pattern général utilisé à chaque migration (ex. Angular N → Angular N+1)

1- Mettre à jour Angular Core et Angular CLI vers la version cible

```
npx ng update @angular/core@<VERSION_CIBLE> @angular/cli@<VERSION_CIBLE>
```

```

sy|fr@LAPTOP-DDG1SG6 MINOW64 ~/Documents/travail/delice-eterne1-gabon/constantine (Integration)
$ npx ng update @angular/cli@18 @angular/core@18
The installed Angular CLI version is outdated.
Installing a temporary Angular CLI versioned 18.2.21 to perform the update.
Installing packages...
✓ Packages successfully installed.
Using package manager: npm
Collecting installed dependencies...
Found 84 dependencies.
Fetching dependency metadata from registry...
Updating package.json with dependency @angular-devkit/architect @ "0.1802.21" (was "0.1703.12")...
Updating package.json with dependency @angular-devkit/build-angular @ "18.2.21" (was "17.3.17")...
Updating package.json with dependency @angular/cli @ "18.2.21" (was "17.3.17")...
Updating package.json with dependency @angular/compiler-cli @ "18.2.14" (was "17.3.12")...
Updating package.json with dependency @angular/language-service @ "18.2.14" (was "17.3.12")...
Updating package.json with dependency @angular/animations @ "18.2.14" (was "17.3.12")...
Updating package.json with dependency @angular/common @ "18.2.14" (was "17.3.12")...
Updating package.json with dependency @angular/compiler @ "18.2.14" (was "17.3.12")...
Updating package.json with dependency @angular/core @ "18.2.14" (was "17.3.12")...
Updating package.json with dependency @angular/forms @ "18.2.14" (was "17.3.12")...
Updating package.json with dependency @angular/localize @ "18.2.14" (was "17.3.12")...
Updating package.json with dependency @angular/platform-browser @ "18.2.14" (was "17.3.12")...
Updating package.json with dependency @angular/platform-browser-dynamic @ "18.2.14" (was "17.3.12")...
Updating package.json with dependency @angular/router @ "18.2.14" (was "17.3.12")...
UPDATE package.json (4604 bytes)
> Cleaning node modules directory
✓ Cleaning node modules directory
> Installing packages
✓ Installing packages
** optional migrations of package '@angular/cli' **

this package has 1 optional migration that can be executed.
> Migrate application projects to the new build system.
  ng update @angular/cli --name use-application-builder

** Executing migrations of package '@angular/core' **

> Updates two-way bindings that have an invalid expression to use the longform expression instead.
  Migration completed (No changes made).

> Replace deprecated HTTP related modules with provider functions.
  UPDATE src/app/auth/store/auth.actions.ts (2711 bytes)
  UPDATE src/app/auth/store/auth.reducer.ts (1914 bytes)
  UPDATE src/app/shared/services/api/auth.service.ts (1237 bytes)
  UPDATE src/app/features/store/items.actions.ts (3180 bytes)
  UPDATE src/app/core/interceptors/api.interceptor.ts (2314 bytes)
  UPDATE src/app/core/interceptors/assets.interceptor.ts (676 bytes)
  UPDATE src/app/core/core.module.ts (2673 bytes)
  UPDATE src/app/auth/signup/signup.component.ts (5341 bytes)
  UPDATE src/app/app.module.ts (3036 bytes)
  UPDATE src/app/auth/main/main.component.spec.ts (1394 bytes)
  UPDATE src/app/auth/signin/signin.component.spec.ts (1408 bytes)
  UPDATE src/app/shared/components/not-found/not-found.component.spec.ts (1112 bytes)
  UPDATE src/app/shared/services/permissions/permission.service.spec.ts (710 bytes)
  UPDATE src/app/shared/services/roles/roles.service.spec.ts (779 bytes)
  Migration completed (14 files modified).

> updates calls to afterRender with an explicit phase to the new API.
  Migration completed (No changes made).

> Adds 'bootstrapcontext' to 'bootstrapApplication' calls in 'main.server.ts' to support server rendering.
  Migration completed (No changes made).

```

2- Mettre à jour les bibliothèques Angular associées si présentes (exemples fréquents : Material/CDK, Google Maps, etc.)

```
npx ng update @angular/material@<VERSION_CIBLE> @angular/cdk@<VERSION_CIBLE>
```

3- Mettre à jour les bibliothèques fortement liées à Angular (par exemple NgRx, qui dépend fortement de la version Angular)

```
px ng update @ngrx/store@<VERSION_CIBLE> @ngrx/effects@<VERSION_CIBLE>
@ngrx/store-devtools@<VERSION_CIBLE>
```

4- Réinstaller proprement les dépendances en cas de conflits ou d'erreurs npm:

Cette étape a souvent été nécessaire lors des migrations majeures, afin de repartir sur un environnement propre.

```
rm -rf node_modules package-lock.json # Suppression des repertoire générés
```

```
npm cache clean --force # Suppression des donnée du cache
```

```
npm install # reinstallation de l'application en local
```

5- Vérifier la compilation et le démarrage de l'application

Après chaque montée de version, l'application était systématiquement relancée pour détecter rapidement les erreurs.

```
npm start # Démarrage de l'application en local
```

```
# ou
```

```
ng serve
```

6- Corriger les erreurs puis valider avant de passer à la version suivante

Cette étape consistait à corriger les incompatibilités (TypeScript, Firebase, NgRx, i18n, etc.), puis à refaire un test complet. Une fois la version stable, la migration pouvait continuer vers la version suivante.

2.4 Apport de l'intelligence artificielle dans la migration

L'intelligence artificielle a été utilisée comme un véritable assistant technique tout au long de la migration. Elle a permis d'analyser rapidement des messages d'erreur complexes, parfois difficiles à interpréter pour un développeur débutant, et de proposer des solutions adaptées compréhensibles. Ci-dessous, quelques exemples d'erreur que l'IA m'a aidé à résoudre :

Type d'erreur rencontrée	Problème observé	Cause identifiée	Solution proposée (IA)
Firebase / Firestore	Erreur Query / CollectionReference	Mélange entre Firebase compat et SDK modulaire	Suppression des imports compat et utilisation exclusive de l'API Firebase modulaire
NgRx	@Effect non reconnu	Décorateur @Effect déprécié	Migration vers createEffect

Angular	Composant standalone dans NgModule	Composants générés automatiquement en standalone	Ajout de standalone: false ou refactorisation
npm / dépendances	ERESOLVE dependency tree	Incompatibilité entre versions Angular, NgRx et AngularFire	Alignement progressif des versions
Internationalisation	Affichage des clés de traduction	TranslateLoader mal configuré	Correction de la factory de chargement des fichiers JSON

Mon tuteur m'a fortement conseillé d'utiliser l'intelligence artificiel pour m'aider à effectuer cette migration. Toutes les mises à jour proposées par chatgpt étaient systématiquement soumises à la validation de mon tuteur de stage après les avoir testé.

2.5 Firebase et notion de SDK modulaire

Un SDK modulaire est une nouvelle manière d'utiliser une bibliothèque JavaScript. Contrairement à l'ancienne implémentation, où l'ensemble de la bibliothèque était importé globalement, le SDK modulaire permet d'importer uniquement les fonctionnalités réellement utilisées.

Dans l'application initiale (Angular 12), Firebase était utilisé via le mode compat. Ce mode repose sur une API globale et orientée objet, aujourd'hui dépréciée. La migration vers le SDK Firebase modulaire a été indispensable pour assurer la compatibilité avec Angular 21.

Ancienne implémentation (Firebase compat)	Nouvelle implémentation (SDK modulaire)	Avantages apportés
<code>firebase.firestore().collection('users')</code>	<code>collection(firestore, 'users')</code>	Code plus clair, plus moderne
Chargement global de Firebase	Chargement à la demande	Amélioration des performances
API dépréciée	API recommandée	Compatibilité long terme
Risque de conflits	Architecture cohérente	Réduction des erreurs

2.6 Environnement technique et outils utilisés

L'application repose sur plusieurs outils et bibliothèques modernes, présentés dans le tableau ci-dessous avec leur rôle et des exemples concrets d'utilisation.

Outil	Rôle dans le projet	Exemples d'utilisation
Angular CLI	Gestion du projet Angular	ng serve, ng update, ng build
Firebase / AngularFire	Backend et authentification	Connexion utilisateur, Firestore, stockage
NgRx	Gestion de l'état global	Authentification, panier, produits
Angular Material	Interface utilisateur	Menus, boutons, navigation
ngx-translate	Internationalisation	Gestion multilingue de l'application

2.7 Conclusion

Cette migration d'Angular 12 vers Angular 21 a été un travail long, progressif et très formateur. Elle a permis d'acquérir une compréhension approfondie d'Angular, de Firebase et des architectures modernes. L'intelligence artificielle a été un soutien déterminant pour analyser les erreurs complexes et proposer des solutions adaptées. Ce projet illustre l'importance d'une méthodologie rigoureuse, d'une veille technologique constante et de l'utilisation raisonnée des outils modernes.